2026年7月6日

OshiruZine-AI

人工知能・機械学習・LLMを中心に、
技術トレンドから実用事例まで幅広く扱う技術系雑誌。

OshiruZine-AI

2026年7月6日 号

目次

1. Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス　*スタンダード（約2000字）*
2. マルチモーダルAIエージェントの現在地――自律型ワークフローはどこまで実用化されたか　*特集（約4000字）*
3. EU AI Act完全施行から半年――日本企業が今すぐ対応すべきコンプライアンス要件　*スタンダード（約2000字）*
4. オンデバイスSLM最前線――スマホで動く小規模言語モデルの選び方　*ショート（500〜1000字）*

Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス

2026年も後半に突入し、AIコーディングアシスタントはもはや「試してみる」フェーズを完全に脱した。とりわけ、Anthropicの**Claude Code**とOpenAIの**Codex**は、エンタープライズ開発の現場で日常的に使われるインフラへと進化している。しかし、ツールの進化が速すぎるあまり、「なんとなく使っている」開発者と「構造的に使いこなしている」開発者の間に、生産性の格差が広がり始めているのも事実だ。本コラムでは、2026年7月時点の最新動向を踏まえ、これら2大AIコーディングツールを最大限に活用するためのベストプラクティスを整理する。

Claude Code 4.0とCodex Agentの現在地

2026年前半の大きなトピックは、Claude Code 4.0（2026年5月GA）における**マルチファイル自律編集モード**の安定化と、Codex側の**Codex Agent**のリアルタイムコラボレーション機能の正式リリースだった。Claude Codeはターミナルネイティブのまま、プロジェクト全体のコンテキストを200Kトークン超のウィンドウで把握しながら、複数ファイルにまたがるリファクタリングを一気に実行できるようになった。一方のCodex Agentは、GitHub連携の深化により、Issue → ブランチ作成 → 実装 → PR作成までを一貫して自動化するワークフローが実用レベルに達している。

どちらを選ぶかではなく、**両者の特性を理解し、タスクに応じて使い分ける**――**あるいは併用する**ことが、2026年型の開発スタイルだ。

プロンプト設計：「指示」から「仕様書」へ

最も重要なベストプラクティスの変化は、プロンプトの位置づけが「自然言語での指示」から「機械可読な仕様書」へとシフトしている点だ。

Claude Codeでは、プロジェクトルートに配置する`CLAUDE.md`が事実上の"AIへの開発規約"として機能する。2026年のベストプラクティスとして推奨されるのは以下の構成だ。

- **アーキテクチャ概要**：使用フレームワーク、ディレクトリ構成、主要な設計判断の背景
- **コーディング規約**：命名規則、エラーハンドリング方針、禁止パターン
- **テスト戦略**：テストフレームワーク、カバレッジ目標、モック方針
- **セキュリティ制約**：扱ってはいけないデータ、許可されていない外部通信

Codex側でも同様に、リポジトリの`AGENTS.md`やシステムプロンプト設定で同等の文脈を与えることが標準化されつつある。重要なのは、**曖昧な日本語の指示ではなく、構造化された制約と期待値を明示すること**だ。これにより、生成コードの品質は劇的に安定する。

テスト駆動でAIを「検証可能」にする

AIが生成するコードの最大の課題は、一見正しく見えるが微妙にバグを含む——いわゆる**"plausible but wrong"問題**だ。これに対する2026年のコンセンサスは明確で、**テスト駆動AI開発（TDD with AI）**がデファクトになりつつある。

具体的なワークフローはこうだ。

1. 人間がテストケース（期待する入出力、境界条件）を先に書く
2. AIにテストを通す実装を生成させる
3. AIにさらにエッジケースのテストを追加させる
4. 人間がテストの妥当性をレビューする

Claude Codeの`--test-first`フラグ（v4.0で追加）を使えば、生成→テスト実行→修正のループを自動で回すことができる。Codex Agentも、CI連携によりPR作成時に自動テストを通過するまで自己修正を繰り返す設定が可能だ。**AIにコードを書かせる前に、まず「正解の定義」を与える**。この順序を守るだけで、手戻りは大幅に減少する。

コードレビュー：人間の役割の再定義

AI生成コードが増えるほど、コードレビューの重要性はむしろ高まっている。ただし、レビューの焦点は変わった。構文ミスや単純なロジックエラーはAI自身が検出・修正できるため、人間のレビュアーが集中すべきは以下の3点だ。

- **設計判断の妥当性**：AIは与えられた制約内で最適化するが、ビジネス要件の変化や将来の拡張性まで考慮しているとは限らない
- **セキュリティ上の見落とし**：特にClaude Code 4.0ではサンドボックス実行が強化されたが、生成コード内のインジェクション脆弱性やデータ漏洩リスクは人間が確認すべき領域だ
- **依存関係の適切さ**：AIは既知のライブラリを積極的に利用するが、ライセンス互換性やメンテナンス状況まで最新の判断はできない

2026年の先進的なチームでは、「**AIが書いたコードをAIがレビューし、人間はそのレビュー結果を監査する**」という三層構造を採用するケースも増えている。Claude CodeとCodexを交差的にレビューに使う「クロスAIレビュー」も、バイアス軽減策として注目されている手法だ。

セキュリティとガバナンス：企業導入の必須条件

企業でのAIコーディングツール導入が本格化する中、2026年前半には複数のインシデントが業界を揺るがした。あるプロジェクトでAIが生成したコードに含まれていた非公開APIの無断呼び出しが問題となり、**AIコード生成に対するガバナンスフレームワーク**の整備が急速に進んでいる。

実務上のチェックリストとしては、以下が最低限必要だ。

- 生成コードのライセンススキャン（SPDX準拠のSBOM生成との連携）
- プロンプトログの保全（いつ、誰が、どのような指示でコードを生成したか）
- 機密データのプロンプト混入防止（Claude Codeのローカルモード活用、Codexのデータ境界設定）
- 生成コードに対する静的解析の義務化

まとめ：AIは「ジュニア開発者」から「シニアペア」へ

1年前まで「優秀なジュニア開発者」と形容されていたAIコーディングアシスタントは、2026年半ばの今、適切に運用すれば**シニアエンジニアの思考パートナー**として機能するレベルに到達している。ただし、その性能を引き出せるかどうかは、仕様書としてのプロンプト設計、テスト駆動のワークフロー、人間によるレビューの質、そしてガバナンス体制という**周辺の仕組み**にかかっている。

ツールの性能が上がるほど、使う側のプラクティスが問われる——2026年後半は、その差が成果として顕在化するフェーズになるだろう。

マルチモーダルAIエージェントの現在地——自律型ワークフローはどこまで実用化されたか

「AIに仕事を任せる」というフレーズが、ついにマーケティング用語ではなく"設計判断"として語られる時代になった。2026年前半、Google、OpenAI、Anthropicの三社が相次いでマルチモーダル対応のAIエージェント基盤を本格投入し、テキスト・画像・音声・コード実行・GUI操作を横断する自律型ワークフ

ローが現実のプロダクション環境に入り始めている。本稿では、各社プラットフォームの機能比較、実際の業務自動化における導入事例、そしてエージェント設計のベストプラクティスと残る課題を整理する。今まさに導入判断を迫られているエンジニア・アーキテクトに向けて、できる限り実用的な指針を示したい。

2026年前半のランドスケープ——三社の基盤は何が違うのか

まず、現時点で利用可能な主要エージェント基盤の全体像を押さえておこう。

Google — Project Mariner / Gemini 2.5 Agent Mode

Googleは2025年末にプレビュー公開したProject Marinerを、2026年Q1にGemini 2.5のAgent Modeとして正式統合した。最大の特徴はChromeブラウザとのネイティブ統合によるWeb操作能力だ。DOMツリーの直接解析とスクリーンショットベースのビジョン認識を併用するハイブリッドアプローチにより、SPA（Single Page Application）やShadow DOMを含む複雑なWebアプリケーションでも高い操作成功率を実現している。Google Workspaceとの深い統合も強みで、Gmail・スプレッドシート・カレンダーを横断するワークフローを自然言語の指示だけで組み立てられる。Vertex AI上でのエンタープライズ展開も整備が進み、VPC Service Controls内でのエージェント実行がGA（一般提供）となった。

OpenAI — Operator / GPT-5 Agents SDK

OpenAIは2025年初頭にOperatorを限定公開した後、2026年に入ってGPT-5ベースのAgents SDKとしてフルリニューアルした。Computer Use（画面操作）、Code Interpreter（コード実行）、Web Browsing、Function Callingを統一的なツールインターフェースとして扱える点が設計上のコアだ。特筆すべきはHandoffsと呼ばれるマルチエージェントオーケストレーション機構で、専門性の異なるサブエージェントにタスクを委譲し、結果を集約するパターンを宣言的に記述できる。Responses APIを通じてストリーミングで中間結果を返せるため、長時間タスクにおけるユーザーの監視・介入も比較的容易だ。

Anthropic ― Claude Agent / Computer Use 2.0

Anthropicは2025年に先行してComputer Use機能をベータ公開し、2026年Q2にClaude 4 Opus搭載のClaude Agentとして全面刷新した。同社のアプローチはコンスティテューショナルAI（Constitutional AI）の原則をエージェント行動にまで拡張している点でユニークだ。エージェントが取るアクションに対して事前定義されたセーフティポリシーが自動適用され、破壊的操作（ファイル削除、本番DBへの書き込みなど）の前に必ず確認ステップが挿入される。この設計思想により、Anthropicのエージェントは「安全性を優先する環境」での採用が特に進んでいる。また、拡張思考（Extended Thinking）機能により、複雑な推論過程を透明化できる点も、監査要件の厳しいエンタープライズ領域で評価されている。

三社を俯瞰すると、Googleは「エコシステム統合」、OpenAIは「オーケストレーションの柔軟性」、Anthropicは「安全性と透明性」にそれぞれ軸足を置いていることがわかる。どれが"最強"かという問いは無意味で、自社のワークロードと制約に照らして選定すべきフェーズに入っている。

実際の導入事例――どこで使われ、何が自動化されているか

理屈はわかった、では実際に動いているのか。2026年上半期に公開された事例や筆者が取材した内容から、代表的なパターンを紹介する。

事例1：ECサイトのカスタマーサポート自動化（小売業・国内大手）

顧客からの問い合わせメールを受信すると、エージェントが注文管理システム（Webアプリ）にログインし、注文状況を画面操作で確認、返品ポリシーをナレッジベースから検索し、回答ドラフトを生成してオペレーターの承認キューに入れる。従来のRPA＋チャットボットの組み合わせでは、UI変更のたびにシナリオの再構築が必要だったが、ビジョンベースの画面操作により保守コストが大幅に削減されたという。OpenAIのAgents SDKを採用し、問い合わせ分類→情報取得→回答生成をHandoffsで分離した設計が功を奏している。

事例2：金融機関のレポート自動生成（証券会社）

アナリストが自然言語で「直近四半期のセクター別パフォーマンスをまとめてほしい」と指示すると、エージェントがBloomberg端末のデータAPIからデータを取得し、Pythonでグラフを生成、所定のテンプレートに沿ったPDFレポートを出力する。ここではAnthropicのClaude Agentが使われており、拡張思考によって分析ロジックの過程が可視化されるため、コンプライアンス部門のレビューが容易になった。ただし、Bloomberg端末との接続部分は自社開発のMCP（Model Context Protocol）サーバーを介しており、ここの構築・運用には相応のエンジニアリングコストがかかっている。

事例3：SREの障害対応初動自動化（SaaS企業）

監視アラートが発火すると、エージェントがGrafanaダッシュボードのスクリーンショットを取得・分析し、関連するログをElasticsearchから検索、過去の類似インシデントのポストモーテムをConfluenceから引き出して初動対応の推奨手順をSlackに投稿する。Google Gemini Agent Modeを使い、各ツールのAPIとブラウザ操作を組み合わせている。完全自動復旧まではまだ到達していないが、オンコールエンジニアのMTTR（平均復旧時間）が約40%短縮されたとの報告がある。

これらの事例に共通するのは、**エージェントが「完全自律」で業務を終えるケースはまだ少なく、人間の承認・確認をループに含む「ヒューマン・イン・ザ・ループ」設計が主流である**という点だ。これは制約ではなく、現段階における合理的な設計判断である。

エージェント設計のベストプラクティス

実際にエージェントシステムを構築する際のベストプラクティスを、現場から見えてきた知見に基づいて整理する。

1. ツール選択の粒度を適切に設計する

エージェントに渡すツール（Function）の粒度は、細かすぎても粗すぎてもパフォーマンスが落ちる。例えば「データベースに任意のSQLを実行する」という汎用ツールは柔軟だが、エージェントの幻覚（ハルシネーション）リスクが高い。一方、「特定のテーブルからユーザーIDで検索する」という専用ツールは安全だが、想定外のクエリに対応できない。実践的には、**ドメイン固有の高頻度操作を専用ツールとして定義し、低頻度・探索的な操作にはサンドボックス化された汎用ツールを用意する**二層構造が有効だ。

2. 画面操作とAPI呼び出しを適材適所で使い分ける

マルチモーダルエージェントの大きな売りはGUI操作能力だが、APIが存在するサービスに対してわざわざ画面操作で対話するのはレイテンシとエラー率の両面で不利だ。画面操作は「APIが存在しない」「APIでは取得できない情報がUIにある」「UIの操作手順そのものが業務要件（証跡としてスクリーンショットが必要、など）」といったケースに限定するのが現実的だ。

3. マルチエージェント構成は「必要になってから」導入する

2026年のエージェントフレームワーク界隈では、複数のエージェントを協調させるマルチエージェントアーキテクチャが注目されている。しかし、安易にエージェントを分割すると、エージェント間の通信オーバーヘッド、コンテキストの喪失、デバッグの困難さが指数関数的に増大する。まずは単一エージェント＋複数ツールの構成で始め、単一エージェントのコンテキストウィンドウやレスポンス時間が限界に達した段階で初めてマルチエージェント構成を検討する、という段階的アプローチを推奨する。

4. 観測可能性（Observability）を初日から組み込む

エージェントの動作は非決定的であり、同じ入力に対して異なるツール呼び出しシーケンスを取ることがある。このため、各ステップのツール呼び出し・入出力・推論トレースを構造化ログとして記録し、後から検索・分析できる仕組みが不可欠だ。LangSmith、Braintrust、Arize PhoenixといったLLMオブザーバ

ビリティツールの導入を早期段階から検討してほしい。OpenTelemetryとの統合が進んでいるツールを選べば、既存のAPM基盤との連携もスムーズだ。

5. フェイルセーフとロールバックを設計に織り込む

エージェントが外部システムに対して副作用のある操作（メール送信、データベース更新、ファイル削除など）を行う場合、操作の取り消しまたはロールバックが可能な設計にしておくべきだ。具体的には、操作をまず「ドライラン」として実行し、結果のプレビューを人間またはバリデーターエージェントに提示してから確定する二段階コミットパターンが有効である。

残る課題——何がまだ「使えない」のか

楽観的な事例ばかり並べても誤解を招く。現時点で依然として課題となっている領域を正直に挙げておく。

長時間タスクの信頼性。 現在のエージェントは、15〜20ステップを超える長いワークフローになるとエラー率が急増する傾向がある。コンテキストウィンドウの制約というよりも、途中のステップでの小さな判断ミスが後続に伝播する「エラー雪崩」が本質的な問題だ。チェックポイントとリカバリの仕組みは各社改善を進めているが、完全解決には至っていない。

コスト。 マルチモーダルエージェントはテキストのみの利用に比べてトークン消費量が桁違いに大きい。画面操作では1アクションごとにスクリーンショットの解析が走り、数千〜数万トークンを消費する。月間数万件の処理を想定すると、API費用だけで相当額になる。コストの予測可能性をどう担保するかは、導入判断における重要な検討事項だ。

セキュリティとガバナンス。 エージェントに社内システムへのアクセス権を付与するということは、LLMの判断ミスや悪意あるプロンプトインジェクションが直接的なセキュリティインシデントにつながるリスクを意味する。最小権限の原則、操作ログの監査、異常検知の仕組みを組織横断で整備する必要があるが、多くの企業ではまだ手探りの状態だ。

評価手法の未成熟。 エージェントの性能を定量的に評価する標準的な手法がまだ確立されていない。タスク成功率、ステップ効率、コスト効率などの指標は提案されているが、業務固有のタスクに対するベンチマーク設計は各社が個別に取り組んでいる段階だ。

今、導入判断をするエンジニアへの指針

最後に、2026年7月というこのタイミングで導入を検討しているエンジニアに向けて、実用的な指針をまとめる。

まず「人間が繰り返しやっている、手順が比較的明確な、ミスのリカバリが容易なタスク」から始めよ。 カスタマーサポートの情報収集、定型レポート生成、テスト環境でのデータセットアップなどが好適だ。逆に、本番環境への直接操作、金銭に関わるトランザクション、法的リスクの高い判断は、十分な検証なしにエージェントに委ねるべきではない。

プラットフォームの選定は、既存のエコシステムとの親和性を第一基準にせよ。 Google Workspaceが中心ならGemini Agent Mode、OpenAI APIを既に大規模に利用しているならAgents SDK、安全性・透明性に対する組織的要件が厳しいならClaude Agent、というのが現時点での大まかな指針だ。

そして、エージェントを「銀の弾丸」ではなく「新しい種類のソフトウェアコンポーネント」として扱え。 テスト戦略、監視、障害対応、セキュリティレビューといった従来のソフトウェアエンジニアリングの規律をそのまま適用することが、結局は最も確実な成功への道筋だ。

マルチモーダルAIエージェントは、確かに「使える」段階に入った。しかし、「何でも任せられる」段階にはまだ距離がある。その距離を正確に測り、現時点

で合理的な範囲で活用すること――それこそが、2026年のエンジニアに求められるスキルだと筆者は考えている。

EU AI Act完全施行から半年――日本企業が今すぐ対応すべきコンプライアンス要件

「うちはEU域内にオフィスがないから関係ない」――もしあなたのチームにそう考えている人がいたら、今すぐ認識をアップデートしてほしい。2026年2月2日、EU AI Actの高リスクAIシステムに関する規制が全面適用となり、施行からすでに半年が経過した。この間、欧州AI庁（AI Office）は初のガイドライン改訂を公表し、域外適用の具体的な執行方針を明確化した。日本国内でAIを開発していても、そのシステムがEU域内のユーザーに影響を及ぼすならば規制の射程に入る。グローバル展開を見据える日本の開発チームにとって、「対岸の火事」ではいられないフェーズに突入している。

EU AI Actの現在地――2026年前半に何が起きたか

EU AI Actは段階的に施行が進められてきた。2024年8月の発効後、まず2025年2月に禁止AIプラクティス（ソーシャルスコアリング、リアルタイム遠隔生体識別の無制限利用など）の規制が適用開始。そして2026年2月、最も実務インパクトの大きい**高リスクAIシステム（Annex III）への義務**が全面適用となった。

2026年前半の注目動向として、欧州AI庁が5月に公開した「高リスクAIシステムの適合性評価に関する実務ガイダンスv2.0」がある。ここではリスク管理システムの具体的な実装水準、技術文書のテンプレート、さらにログ保持期間の推奨値（最低6か月、領域によっては最大10年）が示された。加えて、6月にはフランス

のCNILとドイツのBfDIが共同で、域外事業者に対する初の正式な情報提供要請（Request for Information）を複数社に送付したと報じられている。執行は「いつか来る話」ではなく、すでに動き始めているのだ。

日本企業が直面する3つの技術的対応ポイント

高リスクAIシステムを開発・提供する日本企業が、実装レベルで対処すべき要件を3つに整理する。

1. ログ管理——「後から検証できる」設計の徹底

EU AI Act第12条は、高リスクAIシステムに対して**自動ログ記録機能（automatic logging）**の実装を義務づけている。記録すべきは、入力データの特性、推論結果、システムの動作状態、そして人間による監視介入の履歴だ。

実務的には、MLOpsパイプラインにおけるモデルのバージョン管理、推論リクエスト/レスポンスのイミュータブルなログ保存、そしてデータリネージの追跡が最低限求められる。MLflowやWeights & Biasesなどの実験管理ツールに加え、本番推論ログをクラウドストレージ上にWORM（Write Once Read Many）形式で保持するアーキテクチャが現実的な選択肢になる。ログの保持期間はシステムの想定ライフサイクルに合わせて設定し、最低でもガイダンスが推奨する6か月を確保しておきたい。

2. リスク評価——継続的なリスク管理システムの構築

第9条が定める**リスク管理システム（Risk Management System）**は、一度アセスメントを実施して終わりではない。設計・開発・デプロイ・運用の全ライフサイクルを通じた**反復的プロセス**として維持・更新する必要がある。

具体的には、以下のサイクルを組織に定着させることが重要だ。

1. **リスク特定**：バイアス、誤分類、悪用可能性、セキュリティ脆弱性などを体系的に洗い出す

2. **リスク推定・評価**：発生確率と影響度を定量的に評価し、許容水準と照合する
3. **リスク軽減策の実装**：データ品質管理、フェアネス指標の継続監視、フォールバック機構の設計
4. **残留リスクの文書化と監視**：軽減後もなお残るリスクを明文化し、モニタリングを続ける

2026年6月に欧州標準化機構（CEN/CENELEC）が公開したハーモナイズド標準のドラフト（prEN 17900シリーズ）は、ISO/IEC 42001（AI管理システム）やISO/IEC 23894（AIリスクマネジメント）との整合を強く意識した構成になっている。これらの国際規格をベースに社内プロセスを整備しておけば、EU適合性評価への対応と同時に、国内で議論が進むAI基本法への備えにもなる。

3. 透明性確保——ユーザーと当局に「説明できる」仕組み

第13条の透明性要件は、高リスクAIシステムの利用者（deployer）がシステムの出力を適切に解釈し、人間による監視（human oversight）を実効的に行えるよう、十分な情報を提供することを求めている。

技術的には、SHAPやLIMEといった説明可能AI（XAI）手法の組み込み、意思決定の根拠を自然言語で出力するインターフェース設計、そしてモデルカード（Model Card）やデータシート（Datasheet for Datasets）の作成・公開が有効だ。特にモデルカードには、学習データの概要、性能指標のサブグループ別分解、既知の限界事項を記載し、システム更新のたびにバージョンを改訂する運用を推奨する。

組織的対応——ガバナンス体制を今すぐ整備せよ

技術的対応と同じくらい重要なのが、組織的なガバナンス体制の構築だ。EU AI Actは、高リスクAIシステムの提供者（provider）に対して品質管理システム（QMS）の確立を求めており（第17条）、これは経営層のコミットメントなしには機能しない。

先進的な日本企業では、すでに「AI倫理委員会」を「AIガバナンス委員会」へと改組し、法務・コンプライアンス部門とエンジニアリング部門の橋渡し役として**AIガバナンスオフィサー**を設置する動きが広がっている。2026年4月に経済産業省が改訂した「AI事業者ガイドライン v2.1」でも、リスクベースアプローチに基づくガバナンス体制の整備が強く推奨されており、EU規制対応と国内ガイドラインの両立を意識した体制設計が合理的だ。

今後の展望——2026年後半に向けて

2026年8月には汎用目的AIモデル（GPAI）に関する行動規範（Code of Practice）の最終版が確定する見通しであり、大規模基盤モデルを活用する開発チームにはさらなる対応が必要になる。また、国内では自民党のAI基本法検討PTが秋の臨時国会への法案提出を視野に入れており、EU AI Actとの相互運用性を意識した制度設計が議論されている。

待ちの姿勢は、もはやリスクそのものだ。ログ管理基盤の整備、リスク評価プロセスの定着、透明性を担保するドキュメンテーション——これらを「いつかやる」ではなく「今すぐ始める」ことが、グローバル市場で戦う日本のAI開発チームにとっての最優先事項である。規制をコストではなく、信頼性という競争力に変える発想が、これからのAI開発の分水嶺になる。

オンデバイスSLM最前線——スマホで動く小規模言語モデルの選び方

あなたのポケットに入っているスマートフォンが、つい2年前のGPUサーバーに匹敵する推論性能を持ち始めている——これは誇張ではない。2026年半ば、オ

ンデバイスSLM（Small Language Model）の実用化はいよいよ本格フェーズに入った。

なぜ今、オンデバイスSLMなのか

背景には2つの大きな潮流がある。第一に、**Qualcomm Snapdragon 8 Elite 2**やApple A19 ProのNPU性能がINT4推論で40TOPS超に到達し、7Bクラスのモデルをリアルタイムに動かせるハードウェア基盤が整った点。第二に、EU AI規則の本格施行やAppleの「Private Cloud Compute」拡張に象徴される**プライバシー・バイ・デザイン**の要請だ。医療・金融・教育など、データをデバイス外に出せないユースケースでの需要が急増している。

主要SLM 3モデル比較——どれを選ぶか

2026年7月時点で、エッジ向けに有力な3モデルを整理する。

項目	Gemma 3 4B	Phi-4-mini (3.8B)	Llama 4-Scout 4B
推論速度（Snapdragon 8 Elite 2, INT4）	約38 tok/s	約45 tok/s	約35 tok/s
MMLU精度	72.1	73.5	71.8
ピーク消費電力	約3.2W	約2.8W	約3.5W
ライセンス	Apache 2.0	MIT	Llama Community
マルチモーダル対応	画像＋テキスト	テキスト中心	画像＋テキスト＋音声

Phi-4-miniは推論速度と省電力のバランスに優れ、テキスト中心のチャットやコード補完に最適だ。**Gemma 3 4B**はマルチモーダル対応とオープンライセンス

の組み合わせが魅力で、プロダクト組み込みのハードルが低い。**Llama 4-Scout 4B**は音声入力まで含む統合的なマルチモーダル処理が強みだが、ライセンス条件の精査が必要になる。

モデル選定の3つの勘所

1. **量子化フォーマットを先に決める。** INT4で十分か、精度要件からINT8が必要かで、搭載可能なモデルサイズが倍近く変わる。まずターゲットデバイスのメモリ上限（多くは8〜12GB共有）から逆算しよう。
2. **ランタイムとの相性を検証する。** MediaPipe LLM Inference、ExecuTorch、Qualcomm AI Hubなど、ランタイムごとに最適化済みモデルが異なる。「動く」と「速い」は別問題だ。
3. **ファインチューニング戦略を持つ。** 汎用ベンチマークの数値より、自社ドメインでのLoRA適用後の精度が実運用を左右する。小さいモデルほどドメイン特化の効果が大きい点は覚えておきたい。

まとめ

SLMの選定は「最強モデル探し」ではなく、**デバイス制約・ユースケース・ライセンスの三角形**の中で最適点を見つける作業だ。幸い、2026年のエコシステムは選択肢が十分に揃った。まずは手元の端末で動かしてみることが、最良の第一歩になるだろう。